

Escuela de Ingeniería Electrónica
Licenciatura en Ingeniería Electrónica
EL 3310 Diseño de Sistemas Digitales
I Semestre 2026

**Proyecto 3: Implementación de una
Microarquitectura RISC-V Segmentada con
Forwarding y Hazard Detection**

Profesor

Dr.-Ing. Ronny García Ramírez

Estudiantes:

Karla Arias Arias
Natalie Sánchez Fernández
Marvin Zelaya Luna

1. Introducción

En este proyecto se realizó la conversión de un procesador RISC-V monociclo previamente desarrollado a una arquitectura segmentada de cinco etapas. El propósito de esta modificación fue aumentar el paralelismo en la ejecución de instrucciones y sentar las bases para la incorporación de mecanismos más avanzados de optimización.

Como parte de esta transición, fue necesario implementar soluciones para los principales problemas asociados a las arquitecturas segmentadas, particularmente los hazards de datos y de control. Para ello se incorporaron unidades de detección y resolución de conflictos que garantizan el correcto funcionamiento del pipeline durante la ejecución de programas.

Adicionalmente, se desarrolló una unidad de predicción dinámica de saltos basada en una *Branch History Table* (BHT) con contadores saturados de 2 bits, permitiendo anticipar el comportamiento de las instrucciones de tipo **branch** y reducir las penalizaciones asociadas a los cambios de flujo de ejecución.

Finalmente, el procesador fue validado mediante diversos programas de prueba y un entorno de verificación automatizado, con el objetivo de comprobar tanto la funcionalidad de las instrucciones implementadas como el correcto comportamiento de los mecanismos de segmentación y predicción incorporados.

El código fuente desarrollado para este proyecto, junto con los programas de prueba y documentación complementaria, se encuentra disponible en el repositorio GitHub:

<https://github.com/thatskali/IE-EL3310>

1.1. Objetivo del proyecto

- Conversión de un procesador RISC-V uniclo a una arquitectura segmentada de cinco etapas.
- Implementación de mecanismos para el manejo de hazards de datos y control.
- Validación funcional mediante programas de prueba y testbench automatizado.
- Implementación adicional de una unidad de predicción dinámica de saltos.

2. Arquitectura del Pipeline

El procesador desarrollado utiliza una arquitectura pipeline de cinco etapas: *Fetch (IF)*, *Decode (ID)*, *Execute (EX)*, *Memory (MEM)* y *Write Back (WB)*. Esta organización permite que múltiples instrucciones se ejecuten simultáneamente en diferentes etapas, mejorando el rendimiento respecto a la implementación uniclo.

Para garantizar el funcionamiento correcto del pipeline, se incorporaron mecanismos de manejo de hazards de datos y control, así como una unidad de predicción dinámica de saltos.

2.1. Pipeline de cinco etapas

- **IF (Instruction Fetch):** Obtención de la instrucción desde la memoria de instrucciones.
- **ID (Instruction Decode):** Decodificación de la instrucción y lectura del banco de registros.
- **EX (Execute):** Ejecución de operaciones aritméticas, lógicas y cálculo de direcciones.
- **MEM (Memory):** Acceso a memoria para instrucciones de carga y almacenamiento.
- **WB (Write Back):** Escritura de resultados en el banco de registros.

2.2. Modificación del Registro PC

Como parte de la transición al pipeline, el módulo pc fue modificado para incorporar la señal **StallF** proveniente de la Hazard Unit. En el diseño uniciclo original, el PC se actualizaba en cada flanco de reloj sin excepción. Sin embargo, en la arquitectura segmentada esto ya no es suficiente, porque cuando se detecta una dependencia tipo load-use es necesario congelar las etapas Fetch y Decode para evitar que instrucciones incorrectas avancen por el pipeline.

Para lograr este comportamiento, se agregó una condición que inhibe la actualización del PC mientras **StallF** esté activa. De esta manera, el contador de programa mantiene su valor actual hasta que la dependencia sea resuelta, permitiendo que la instrucción en Decode permanezca estable y que Fetch no obtenga una nueva instrucción antes de tiempo.

```
always_ff @(posedge clk) begin
    if (rst)          PC <= 32'b0;
    else if (!StallF) PC <= PCNext;
end
```

2.3. Registros de Pipeline

Para definir los registros intermedios del pipeline se partió del diagrama general de la microarquitectura segmentada, mostrado en la Figura 1. A partir de este esquema se comenzó a analizar cuáles señales se generaban en cada etapa y cuáles debían pasar o no hacia los registros de pipeline. Este análisis permitió determinar qué información debía conservarse entre etapas consecutivas, evitando propagar señales innecesarias y asegurando que cada etapa recibiera únicamente los datos y señales de control requeridos para su operación.

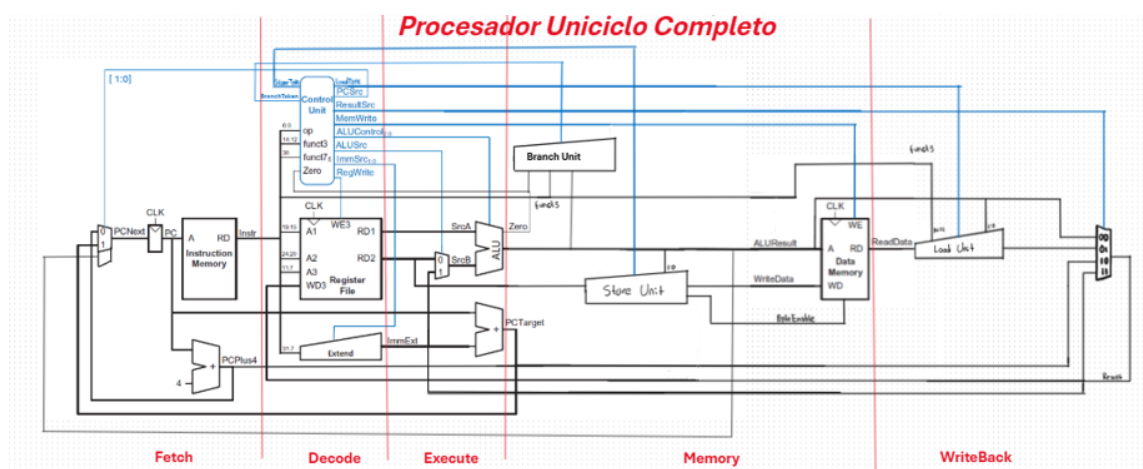


Figura 1: Diagrama utilizado como punto de partida para decidir las señales de entrada y salida de los registros de pipeline.

En general, cada registro se diseñó como una barrera síncrona entre dos etapas consecutivas. En cada flanco activo de reloj, el registro captura las señales generadas por la etapa anterior y las entrega a la etapa siguiente durante el ciclo posterior. Además, algunos registros incorporan señales de control provenientes de la Hazard Unit para congelar o limpiar su contenido cuando se presentan dependencias de datos o cambios en el flujo de control [1].

2.3.1. Registro IF/ID

El registro IF/ID se implementó como una captura del estado al final de la etapa Fetch. En esta etapa se encuentran disponibles el contador de programa actual, la instrucción leída desde

la memoria de instrucciones y el valor **PC+4**. Estas señales deben preservarse porque la etapa Decode necesita la instrucción para identificar los registros fuente y destino, generar el inmediato y producir las señales de control correspondientes. Además, el valor del PC se utiliza para el cálculo de direcciones objetivo, mientras que **PC+4** se conserva para instrucciones de salto que deben escribir la dirección de retorno.

Por esta razón, las entradas del registro corresponden a **PCF**, **InstrF** y **PCPlus4F**, y sus salidas hacia Decode se nombran como **PCD**, **InstrD** y **PCPlus4D**. Este registro también recibe las señales **StallD** y **FlushD**. Cuando **StallD** está activa, el registro mantiene sus valores anteriores para congelar la instrucción en Decode durante un hazard tipo load-use. Cuando **FlushD** está activa, el registro se limpia para descartar una instrucción obtenida incorrectamente debido a un cambio en el flujo de control.

2.3.2. Registro ID/EX

El registro ID/EX almacena la información generada durante Decode que será necesaria en Execute. A partir del diagrama se identificó que esta etapa requiere los operandos leídos del banco de registros, el inmediato extendido, el PC, el valor **PC+4** y los números de registros fuente y destino. Estos campos permiten ejecutar operaciones aritméticas y lógicas, calcular direcciones de memoria, evaluar saltos y proporcionar a la Hazard Unit la información necesaria para detectar dependencias.

También se propagan las señales de control generadas por la unidad de control, tales como **RegWrite**, **MemWrite**, **ALUSrc**, **Jump**, **Branch**, **ResultSrc**, **ALUControl**, **LoadType** y **StoreType**. Este registro no se congela mediante una señal de stall, ya que cuando se detecta una dependencia load-use se congelan Fetch y Decode, mientras Execute recibe una burbuja. Por ello, el registro ID/EX utiliza la señal **FlushE** para poner en cero sus salidas e insertar una operación nula en la etapa Execute.

2.3.3. Registro EX/MEM

El registro EX/MEM conserva los resultados producidos en Execute que serán utilizados durante Memory y, posteriormente, en Writeback. Entre estas señales se incluye **ALUResult**, que puede representar el resultado de una operación aritmética o la dirección efectiva para instrucciones de carga y almacenamiento. En el caso de instrucciones **store**, también se propaga el dato que será escrito en memoria junto con la información necesaria para habilitar los bytes correspondientes.

Además, se conserva el registro destino **RdE**, debido a que debe llegar hasta Writeback para seleccionar el registro que recibirá el resultado final. Las señales de control se propagan solo hasta donde son necesarias: **MemWrite** y las señales asociadas al almacenamiento se utilizan en Memory, mientras que **RegWrite**, **ResultSrc**, **LoadType** y **PCPlus4** continúan hacia la etapa final. Este registro no requiere señales de stall ni flush, porque los congelamientos se aplican en Fetch y Decode, y los vaciados se realizan antes de que la instrucción avance a Memory.

2.3.4. Registro MEM/WB

El registro MEM/WB se encarga de almacenar los valores finales que pueden ser seleccionados para escritura en el banco de registros. En esta etapa se conserva el dato leído desde memoria para instrucciones de carga, el resultado de la ALU, el valor **PC+4** para instrucciones de salto y el registro destino correspondiente. De esta manera, la etapa Writeback cuenta con todas las posibles fuentes de resultado para seleccionar el valor correcto mediante el multiplexor controlado por **ResultSrc**.

También se propagan las señales **RegWrite**, **ResultSrc** y **LoadType**, ya que determinan si se debe escribir en el banco de registros, cuál fuente debe seleccionarse y cómo debe interpretarse el dato cargado desde memoria. Al igual que el registro EX/MEM, este registro no recibe señales

de stall ni flush, debido a que las instrucciones que alcanzan esta etapa ya corresponden a operaciones válidas del flujo de ejecución.

3. Unidad de Control

La unidad de control es uno de los bloques fundamentales del procesador, ya que se encarga de interpretar cada instrucción y generar las señales necesarias para coordinar el funcionamiento del pipeline. Su implementación se dividió en dos módulos principales: el *Main Decoder*, responsable de identificar el tipo general de instrucción, y el *ALU Decoder*, encargado de seleccionar la operación específica que debe realizar la unidad aritmético-lógica.

3.1. Main Decoder

El *Main Decoder* utiliza el campo `opcode` de la instrucción para determinar su categoría y generar las señales de control correspondientes. Estas señales controlan aspectos como la escritura en registros, el acceso a memoria, la selección de operandos para la ALU, la generación de inmediatos y la actualización del contador de programa.

La lógica implementada permite reconocer las instrucciones del conjunto RV32I utilizadas en el proyecto, incluyendo:

- Instrucciones de carga (LB, LH, LW, LBU, LHU).
- Instrucciones de almacenamiento (SB, SH, SW).
- Instrucciones aritméticas y lógicas tipo R.
- Instrucciones aritméticas y lógicas tipo I.
- Instrucciones de salto condicional (BEQ, BNE, BLT, BGE).
- Saltos incondicionales (JAL).
- Saltos indirectos (JALR).
- Instrucción LUI.

A partir de esta decodificación se generan señales como `RegWrite`, `MemWrite`, `ALUSrc`, `ResultSrc`, `ImmSrc` y `PCSrc`, las cuales determinan el comportamiento de las diferentes etapas del pipeline durante la ejecución de cada instrucción.

3.2. ALU Decoder

Una vez identificado el tipo general de instrucción, el *ALU Decoder* analiza los campos `funct3` y `funct7` para determinar la operación exacta que debe ejecutar la ALU. Este enfoque permite reutilizar una misma ruta de datos para múltiples instrucciones, diferenciándolas únicamente mediante las señales de control generadas.

Las operaciones soportadas por la ALU incluyen:

- Suma (ADD, ADDI).
- Resta (SUB).
- Operaciones lógicas (AND, OR, XOR).
- Desplazamientos lógicos (SLL, SRL).

- Desplazamientos aritméticos (SRA).
- Comparaciones con signo (SLT).
- Comparaciones sin signo (SLTU).

La separación entre el *Main Decoder* y el *ALU Decoder* permitió mantener una estructura modular y fácilmente extensible, facilitando la incorporación de nuevas instrucciones y simplificando la lógica de control del procesador.

4. Manejo de Hazards

En una arquitectura segmentada, se ejecutan simultáneamente múltiples instrucciones en diferentes etapas del pipeline. Esta superposición puede generar conflictos conocidos como hazards, los cuales deben resolverse para garantizar la correcta ejecución del programa. Para este proyecto se implementaron mecanismos de resolución de data hazards y control hazards.

4.1. Data Hazards

Los data hazards ocurren cuando una instrucción requiere un operando que aún no ha sido escrito por una instrucción anterior. Para minimizar la pérdida de rendimiento, se implementaron mecanismos de forwarding y detección de dependencias tipo load-use.

4.1.1. Forwarding

El mecanismo de forwarding permite redirigir resultados generados en etapas posteriores del pipeline directamente hacia las entradas de la ALU, evitando ciclos de espera innecesarios.

La Hazard Unit compara los registros fuente de la instrucción ubicada en la etapa Execute (Rs1E y Rs2E) con los registros destino de las etapas posteriores (RdM y RdW). Cuando existe una coincidencia válida y la instrucción correspondiente realiza escritura en el banco de registros, se selecciona el dato más reciente mediante señales de control para los multiplexores de forwarding.

Se implementaron los siguientes casos:

- Forwarding MEM→EX.
- Forwarding WB→EX.

La prioridad se asigna al dato proveniente de la etapa MEM, ya que corresponde al resultado más reciente disponible. Adicionalmente, el registro x0 se excluye del proceso de forwarding debido a que su valor permanece constantemente en cero.

4.1.2. Load-Use Hazard

Las dependencias generadas por instrucciones de carga representan un caso especial que no puede resolverse únicamente mediante forwarding. Esto ocurre porque el dato solicitado desde memoria no se encuentra disponible cuando la siguiente instrucción intenta utilizarlo.

Para detectar esta situación, la Hazard Unit verifica si alguno de los registros fuente de la etapa Decode coincide con el registro destino de la instrucción presente en Execute mientras esta corresponde a una operación de carga (lw).

Cuando se detecta esta condición, se inserta una burbuja en el pipeline mediante las señales:

- StallF
- StallD

- **FlushE**

Las señales **StallF** y **StallD** congelan temporalmente las etapas Fetch y Decode, mientras que **FlushE** elimina la instrucción que avanzaría incorrectamente hacia Execute. De esta forma se garantiza que el dato cargado esté disponible antes de ser utilizado por instrucciones dependientes.

4.2. Control Hazards

Los control hazards aparecen cuando una instrucción modifica el flujo normal de ejecución del programa, impidiendo conocer con certeza cuál será la siguiente instrucción válida hasta que se resuelva la condición de salto.

4.2.1. Instrucciones Branch

El procesador soporta las siguientes instrucciones de salto condicional:

- **BEQ**
- **BNE**
- **BLT**
- **BGE**

La condición asociada al branch es evaluada en la etapa Execute. Una vez determinado el resultado de la comparación, se decide si el contador de programa debe continuar secuencialmente o redirigirse hacia la dirección objetivo calculada.

4.2.2. Instrucciones de Salto

Además de los saltos condicionales, se implementó soporte para las instrucciones:

- **JAL**
- **JALR**

Estas instrucciones modifican directamente el flujo de ejecución y requieren mecanismos adicionales para eliminar instrucciones obtenidas incorrectamente.

4.2.3. Mecanismo de Flush

Cuando una instrucción branch, JAL o JALR provoca una modificación del flujo normal del programa, las instrucciones que ya se encuentran en etapas anteriores del pipeline dejan de ser válidas.

Para resolver este problema, la Hazard Unit genera las señales:

- **FlushD**
- **FlushE**

Estas señales limpian los registros de pipeline correspondientes, eliminando instrucciones incorrectas y permitiendo reiniciar la ejecución desde la dirección de destino calculada. De esta forma se garantiza la consistencia del flujo de control dentro del procesador segmentado.

5. Unidad de Predicción Dinámica de Saltos

Durante el desarrollo del procesador segmentado se observó que las instrucciones de salto introducen hazards de control, ya que la dirección correcta del siguiente PC no se conoce inmediatamente, lo que provoca vaciados del pipeline y ciclos desperdiciados cada vez que se toma una decisión incorrecta.

Con el objetivo de reducir esta penalización, se incorporó una unidad de predicción dinámica de saltos que intenta anticipar el comportamiento de las instrucciones **branch** antes de que la condición sea evaluada completamente.

La unidad se implementó mediante una Branch History Table (BHT), compuesta por contadores saturados de 2 bits. Se eligió este esquema debido a su simplicidad de implementación y a que ofrece una mejora significativa respecto a una predicción estática.

Cada entrada de la tabla almacena el comportamiento reciente de un salto y puede encontrarse en uno de cuatro estados: Strong Not Taken, Weak Not Taken, Weak Taken y Strong Taken. De esta forma, una única ejecución atípica no modifica inmediatamente la predicción, permitiendo un comportamiento más estable frente a patrones repetitivos de ejecución.

Cuando un salto es resuelto, el contador asociado se actualiza incrementándose o decrementándose de manera saturada según el resultado real, fortaleciendo o debilitando la predicción para futuras ejecuciones.

Cuadro 1: Estados del contador saturado de 2 bits.

Estado	Significado
00	Strong Not Taken
01	Weak Not Taken
10	Weak Taken
11	Strong Taken

5.1. Implementación

La unidad de predicción dinámica de saltos fue integrada al pipeline con el objetivo de anticipar el resultado de las instrucciones de tipo **branch** y reducir la cantidad de ciclos perdidos debido a hazards de control. El predictor utiliza una Branch History Table (BHT) indexada mediante bits del contador de programa (PC), donde cada entrada almacena un contador saturado de 2 bits que representa el comportamiento reciente de un salto.

5.1.1. Predicción en Fetch

Durante la etapa Fetch, la BHT es consultada utilizando los bits menos significativos del PC. El valor almacenado en la entrada seleccionada determina la predicción realizada para la instrucción de salto.

Si el contador se encuentra en los estados 10 o 11, el salto se predice como tomado. En caso contrario, cuando el estado es 00 o 01, el salto se predice como no tomado. Esta decisión se utiliza para seleccionar de manera anticipada la dirección del próximo PC y permitir que el pipeline continúe ejecutándose sin esperar a que la condición del salto sea evaluada.

5.1.2. Actualización en Execute

La decisión definitiva del salto ocurre en la etapa Execute, donde se evalúa la condición correspondiente (**beq**, **bne**, **blt**, **bge**, entre otras). Una vez conocido el resultado real, el contador asociado en la BHT es actualizado.

Si el salto fue tomado, el contador se incrementa de forma saturada. Por el contrario, si el salto no fue tomado, el contador se decrementa de forma saturada. Este mecanismo permite que

la predicción se adapte dinámicamente al comportamiento observado durante la ejecución del programa.

5.1.3. Detección de Error de Predicción

Para verificar la precisión del predictor, se compara la decisión realizada durante Fetch con el resultado real obtenido en Execute. Una predicción incorrecta se detecta mediante la condición

$$\text{Mispredict} = (\text{PredictedTakenE} \oplus \text{BranchTakenE}) \quad (1)$$

Cuando ocurre una discrepancia, el pipeline descarta las instrucciones que fueron cargadas utilizando la ruta incorrecta mediante las señales de **flush**. Posteriormente, el PC es redirigido hacia la dirección correcta, garantizando que la ejecución continúe con el flujo adecuado del programa.

5.2. Resultados Obtenidos

Durante las pruebas se registraron estadísticas de funcionamiento del predictor:

- Branches evaluados: 9
- Predicciones tomadas correctamente: 3

Asimismo, se observó experimentalmente que el predictor aprende el comportamiento de branches repetitivos, reduciendo la cantidad de flushes necesarios en iteraciones posteriores.

6. Verificación y Pruebas

La validación del proyecto se realizó utilizando el módulo principal `riscv_pipeline`, el cual integra todos los bloques desarrollados durante la implementación del procesador segmentado. Este módulo constituye el nivel superior del diseño y se encarga de interconectar las cinco etapas del pipeline, los registros intermedios, la unidad de control, la ALU, el banco de registros, las memorias de instrucciones y datos, así como los mecanismos de manejo de hazards y la unidad de predicción dinámica de saltos.

A través de este módulo se ejecutan los programas de prueba utilizados para verificar el comportamiento completo del procesador. De esta forma, las pruebas no evalúan únicamente bloques individuales de manera aislada, sino también la correcta interacción entre todas las unidades que conforman la arquitectura segmentada.

Con el fin de validar el correcto funcionamiento del procesador y de los módulos incorporados durante el desarrollo, se diseñaron dos programas de prueba. Cada programa ejercita diferentes grupos de instrucciones y permite verificar tanto la funcionalidad individual de los bloques como su integración dentro del pipeline.

6.1. Programa de Prueba 1

El primer programa fue diseñado para validar las instrucciones básicas del conjunto RV32I y el correcto funcionamiento de las unidades de procesamiento de datos.

Se verificaron los siguientes aspectos:

- Operaciones aritméticas (ADD, SUB, ADDI).
- Operaciones lógicas (AND, OR, XOR).
- Desplazamientos lógicos y aritméticos.

- Comparaciones (SLT, SLTU).
- Instrucciones de carga (LB, LBU, LH, LHU, LW).
- Instrucciones de almacenamiento (SB, SH, SW).
- Instrucción LUI.

La ejecución del programa permitió comprobar el correcto funcionamiento de la ALU, la memoria de datos, las unidades `load_unit` y `store_unit`, así como la escritura y lectura del banco de registros.

6.2. Programa de Prueba 2

El segundo programa fue diseñado para validar el comportamiento de las instrucciones de control de flujo y los mecanismos asociados al manejo de hazards de control.

Se verificaron los siguientes aspectos:

- Branches condicionales (BEQ, BNE, BLT, BGE).
- Saltos incondicionales (JAL).
- Saltos con registro (JALR).
- Actualización correcta del PC.
- Generación de señales de flush.
- Funcionamiento del predictor dinámico de saltos.

Durante la simulación se observó que las instrucciones de salto modificaban correctamente el flujo de ejecución. Asimismo, se verificó que las predicciones realizadas por la BHT eran actualizadas según el resultado real de cada branch y que las señales de flush se activaban únicamente cuando ocurría una predicción incorrecta.

6.3. Testbench Automatizado

Para facilitar la verificación del procesador, se desarrolló un *testbench* automatizado capaz de ejecutar distintos programas de prueba y comparar los resultados obtenidos con los valores esperados.

La ejecución se realiza mediante el script `run_tests.sh`, indicando como argumento el programa que se desea simular:

```
./run_tests.sh programa1  
./run_tests.sh programa2
```

Este *testbench* carga automáticamente el archivo de instrucciones correspondiente, ejecuta la simulación y muestra el contenido final del banco de registros. Además, incluye verificaciones automáticas mediante mensajes PASS/FAIL, lo que permite identificar rápidamente si alguna instrucción o módulo no está funcionando correctamente.

7. Resultados

En esta sección se presentan los resultados obtenidos durante la ejecución de los programas de prueba desarrollados para validar el funcionamiento del procesador segmentado y de la unidad de predicción dinámica de saltos.

7.1. Programa 1

El Programa 1 fue diseñado para validar el funcionamiento de las instrucciones aritméticas, lógicas, de carga, almacenamiento y la instrucción LUI. Para cada operación se calcularon previamente los valores esperados y se almacenaron en registros específicos, permitiendo verificar automáticamente el comportamiento del procesador al finalizar la ejecución.

La Figura 2 muestra el contenido final del banco de registros. Cada valor observado corresponde al resultado esperado de las operaciones ejecutadas durante la simulación. Por ejemplo, los registros `x3` y `x4` contienen el valor `0x7F`, validando las operaciones de carga; mientras que `x5` y `x6` contienen `0x12C`, confirmando el correcto funcionamiento de las operaciones aritméticas y del acceso a memoria. Asimismo, el registro `x2` conserva el valor `0x12345000`, verificando la correcta ejecución de la instrucción LUI.

```
=====
REGISTER FILE FINAL
=====
x0  = 00000000
x1  = 00000064
x2  = 12345000
x3  = 0000007f
x4  = 0000007f
x5  = 0000012c
x6  = 0000012c
x7  = 12345000
x8  = 0000000a
x9  = 00000003
x10 = 0000000d
x11 = 00000007
x12 = 00000009
x13 = 0000000f
x14 = 0000000b
x15 = 0000000b
x16 = 00000002
x17 = 00000002
x18 = 00000018
x19 = 0000000c
x20 = 00000003
x21 = 0000000c
x22 = ffffffff0
x23 = ffffffffe
x24 = ffffffffc
x25 = 00000001
x26 = 00000001
x27 = 00000001
x28 = 00000001
x29 = 00000000
x30 = 00000000
x31 = 00000000
```

Figura 2: Contenido final del banco de registros tras la ejecución del Programa 1.

La validación se realizó mediante un mecanismo automático de comparación entre los valores obtenidos y los valores de referencia definidos para cada registro. Como se observa en la Figura 3, todas las verificaciones reportaron un resultado `PASS`, indicando que los datos calculados por

el procesador coinciden exactamente con los resultados esperados.

```
=====
RESULTADOS FINALES
=====
Verificando programa1...
PASS:                x1 = 00000064
PASS:                x2 = 12345000
PASS:                x3 = 0000007f
PASS:                x4 = 0000007f
PASS:                x5 = 0000012c
PASS:                x6 = 0000012c
PASS:                x7 = 12345000
PASS:                x8 = 0000000a
PASS:                x9 = 00000003
PASS:               x10 = 0000000d
PASS:               x11 = 00000007
PASS:               x12 = 00000009
PASS:               x13 = 0000000f
PASS:               x14 = 0000000b
PASS:               x15 = 0000000b
```

Figura 3: Verificación automática de los resultados obtenidos en el Programa 1.

Adicionalmente, se muestran las estadísticas del predictor dinámico de saltos para esta prueba. Debido a que el Programa 1 únicamente evalúa instrucciones de procesamiento de datos y acceso a memoria, no se ejecutaron instrucciones de tipo **branch**. Por esta razón, el número de branches evaluados y de predicciones realizadas es igual a cero, tal como se aprecia en la Figura 4. Este resultado confirma que el predictor permanece inactivo cuando no existen instrucciones de control de flujo que requieran predicción.

```
=====
BRANCH PREDICTOR STATS
=====
Branches evaluados    = 0
Predicciones tomadas = 0
=====
```

Figura 4: Estadísticas del predictor dinámico de saltos durante la ejecución del Programa 1.

7.2. Programa 2

El Programa 2 fue diseñado para verificar el correcto funcionamiento de las instrucciones de control de flujo del procesador, incluyendo branches condicionales, saltos incondicionales (**JAL**), saltos indirectos (**JALR**) y la integración de la unidad de predicción dinámica de saltos.

La Figura 5 muestra el contenido final del banco de registros al concluir la simulación. Los valores almacenados coinciden con los resultados esperados definidos para cada prueba, permitiendo confirmar que las transferencias de control fueron ejecutadas correctamente y que el contador de programa fue actualizado según la ruta de ejecución prevista.

En particular, los registros **x5**, **x6** y **x7** contienen los valores generados mediante instrucciones **LUI** y **ADDI**, mientras que los registros **x21** y **x22** almacenan resultados utilizados para verificar que únicamente las instrucciones pertenecientes al camino correcto fueron ejecutadas. Esto permite validar indirectamente el correcto funcionamiento de los mecanismos de redirección del flujo de ejecución.

```
=====
REGISTER FILE FINAL
=====
x0  = 00000000
x1  = 00000005
x2  = 00000005
x3  = 00000002
x4  = 00000000
x5  = 12345000
x6  = abcde000
x7  = 12345001
x8  = 00000001
x9  = 00000000
x10 = 00000000
x11 = 00000000
x12 = 00000000
x13 = 00000000
x14 = 00000000
x15 = 00000000
x16 = 00000000
x17 = 00000000
x18 = 00000000
x19 = 00000000
x20 = 00000000
x21 = 00000001
x22 = 00000063
x23 = 00000000
x24 = 00000000
x25 = 00000000
x26 = 00000000
x27 = 00000000
x28 = 00000000
x29 = 00000000
x30 = 00000000
x31 = 00000000
```

Figura 5: Contenido final del banco de registros tras la ejecución del Programa 2.

La validación de la prueba se realizó mediante comparaciones automáticas entre los valores obtenidos y los valores de referencia definidos para cada registro evaluado. Como se observa en la Figura 6, todas las verificaciones finalizaron con estado **PASS**, indicando que las instrucciones **branch**, **JAL** y **JALR** produjeron exactamente los resultados esperados.

```

=====
RESULTADOS FINALES
=====
Verificando programa2...
PASS:                x1 = 00000005
PASS:                x2 = 00000005
PASS:                x3 = 00000002
PASS:                x5 (LUI) = 12345000
PASS:                x6 (LUI) = abcde000
PASS:                x7 (ADDI) = 12345001
PASS:                x8 = 00000001
PASS:                x10 = 00000000
PASS:                x20 = 00000000
PASS:                x21 = 00000001
PASS:                x22 = 00000063

```

Figura 6: Verificación automática de resultados para el Programa 2.

Adicionalmente, la Figura 7 presenta las estadísticas recopiladas por la unidad de predicción dinámica de saltos durante la ejecución del programa. Se observa que fueron evaluadas nueve instrucciones de tipo branch y que el predictor realizó tres predicciones de salto tomado. Estos resultados confirman que la BHT fue utilizada activamente durante la simulación y que el mecanismo de predicción se integró correctamente con el pipeline.

```

=====
BRANCH PREDICTOR STATS
=====
Branches evaluados    = 9
Predicciones tomadas  = 3
=====

```

Figura 7: Estadísticas del predictor dinámico de saltos durante la ejecución del Programa 2.

Los resultados obtenidos demuestran que el procesador ejecuta correctamente instrucciones de control de flujo y que la unidad de predicción dinámica participa activamente en la toma anticipada de decisiones de salto, reduciendo la necesidad de detener el pipeline mientras se resuelve la condición real del branch.

8. Conclusiones

- La migración de una arquitectura uniciclo a una arquitectura segmentada permitió ejecutar múltiples instrucciones de manera simultánea en distintas etapas del pipeline, evidenciando las ventajas de la segmentación como técnica fundamental para mejorar el aprovechamiento de los recursos del procesador.
- Los mecanismos implementados para el manejo de hazards demostraron ser efectivos durante las pruebas realizadas, garantizando la correcta ejecución de las instrucciones aun en presencia de dependencias de datos y cambios en el flujo de control.
- Los resultados obtenidos mediante los programas de prueba confirmaron el correcto funcionamiento de las instrucciones aritméticas, lógicas, de acceso a memoria y de control de flujo, mostrando coincidencia entre los valores esperados y los valores obtenidos al finalizar cada simulación.

- La incorporación de la unidad de predicción dinámica de saltos permitió extender la funcionalidad básica del pipeline, demostrando la capacidad del procesador para anticipar el comportamiento de instrucciones **branch** y adaptarse dinámicamente a patrones repetitivos de ejecución.
- El entorno de verificación automatizado facilitó el proceso de validación del diseño, permitiendo detectar errores de manera rápida y proporcionando evidencia objetiva del correcto funcionamiento de cada uno de los módulos implementados.
- En conjunto, los resultados presentados validan la correcta integración de todos los componentes desarrollados y demuestran el funcionamiento estable del procesador RISC-V segmentado propuesto.

Referencias

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, RISC-V ed. Burlington, MA, USA: Morgan Kaufmann, 2021.

Apéndices

A. Ejecución de Simulaciones

Las pruebas funcionales del procesador fueron ejecutadas mediante un script automatizado que compila el diseño, carga el programa seleccionado y ejecuta la simulación utilizando Icarus Verilog.

Para ejecutar los programas de prueba utilizados durante la validación del proyecto se emplearon los siguientes comandos:

```
./run_tests.sh programa1  
./run_tests.sh programa2
```

El script se encarga de compilar automáticamente todos los módulos del procesador, ejecutar el testbench correspondiente y mostrar los resultados obtenidos en la consola.

De forma alternativa, la simulación puede ejecutarse manualmente utilizando los siguientes comandos:

```
iverilog -g2012 -o simv tb/tb_riscv_pipeline.sv src/*.sv  
vvp simv
```

B. Corrección de la instrucción LUI en el datapath uniciclo

Durante el desarrollo del Proyecto 2 se identificó que la instrucción LUI no había sido implementada correctamente siguiendo el datapath estándar de la arquitectura RISC-V. Si bien el procesador producía el resultado correcto, la solución adoptada en ese momento forzaba la entrada **SrcA** de la ALU a cero mediante una condición especial sobre el **opcode**. Esto representaba una modificación *ad hoc* del datapath, ya que el funcionamiento dependía de una excepción particular y no del flujo descrito en el diagrama original de la arquitectura.

La Figura 8 muestra el punto del datapath utilizado como referencia para analizar la corrección. A partir de esta revisión se concluyó que el inmediato generado para LUI no debía pasar innecesariamente por la ALU, sino seleccionarse directamente como resultado para escritura en el banco de registros.

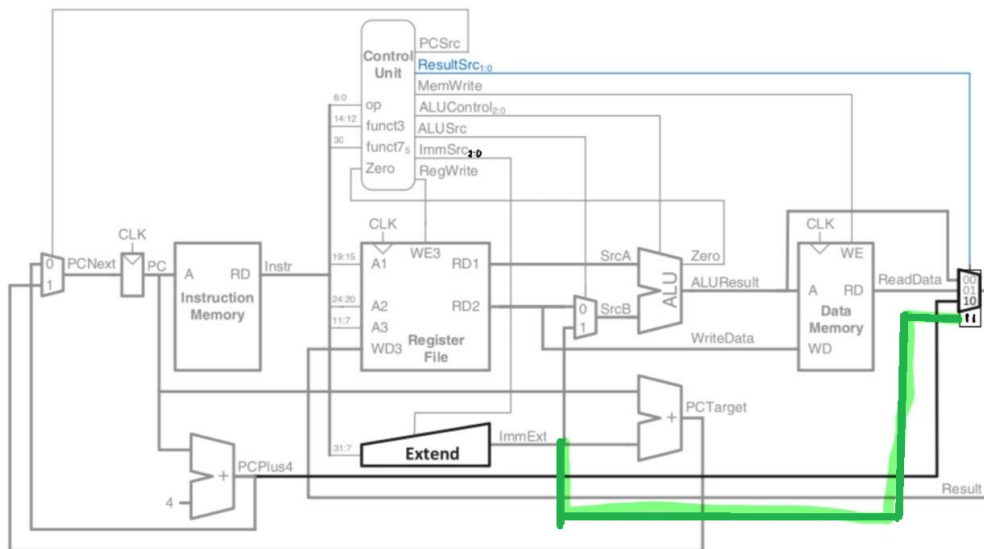


Figura 8: Referencia utilizada para corregir el flujo de la instrucción LUI dentro del datapath.

Para corregir este comportamiento se realizaron tres modificaciones sobre el diseño unificado base. En primer lugar, el multiplexor de resultado `mux31` fue extendido a cuatro entradas, por lo que pasó a denominarse `mux41`. La cuarta entrada recibe directamente la señal `ImmExt` proveniente del bloque `Extend`, permitiendo que el inmediato extendido sea seleccionado como resultado sin necesidad de pasar por la ALU.

En segundo lugar, en el módulo `main_deco` se modificó la señal `ResultSrc` correspondiente a la instrucción LUI, cambiando su valor de `2'b00` a `2'b11`. De esta forma, cuando se ejecuta una instrucción LUI, el multiplexor de resultado selecciona directamente `ImmExt` como valor a escribir en el registro destino.

Finalmente, en el módulo `riscv_top` se eliminó la asignación condicional de `SrcA` que forzaba su valor a cero para instrucciones LUI, reemplazándola por una conexión directa a `RD1`. Con esto se restaura el comportamiento estándar del datapath, donde `SrcA` siempre proviene del banco de registros y no depende de una excepción particular para una instrucción específica.

Con estas correcciones, la instrucción LUI sigue el flujo correcto del datapath: el bloque `Extend` genera el inmediato con los 20 bits superiores indicados por la instrucción y los 12 bits inferiores en cero; luego, este valor llega directamente al banco de registros a través del multiplexor de resultado controlado por `ResultSrc = 2'b11`, sin involucrar innecesariamente la ALU.